

Logic Design and Switching Theory

Lecture – 5

Subtitle: Active Low Logic and NAND/NOR
Implementation

by
Ms. Hera Noor

What's new!

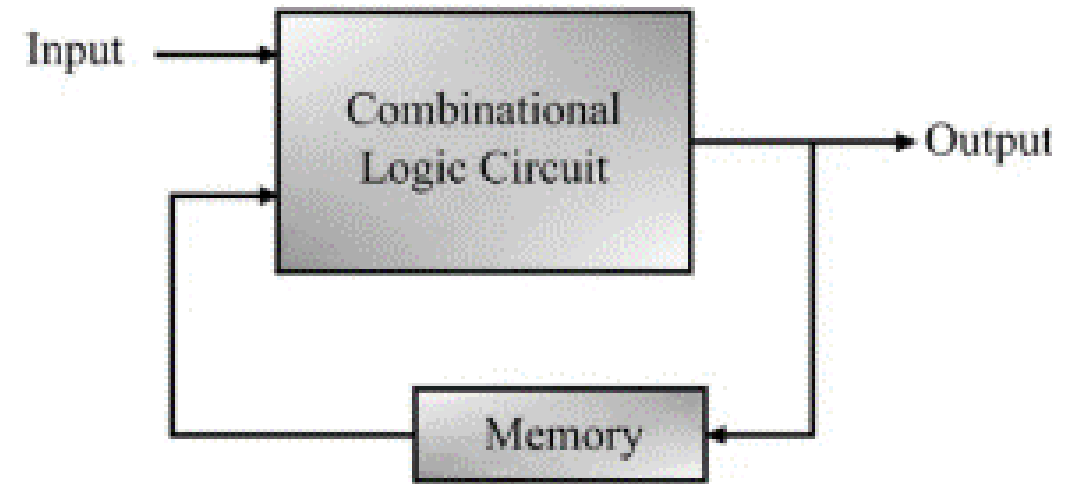
- Practice 5 variable k-map
- Active High & Active Low Logic
- Combinational Logic Circuit Analysis
 - AND-OR Logic
 - AND-OR Invert Logic
 - Exclusive-OR logic
 - Exclusive-NOR Logic
- Universal property of NAND
 - The NAND Gate as a Universal Logic Element
 - The NOR Gate as a Universal Logic Element

Variable Entrant Map

Minimization procedure for VEM

- Step 1: Write all the variables(original and complimented forms are treated as two different variables) in the map as 0, leave 0's, minterms and don't cares as it is and obtain the SOP expression.
- Step 2: Select one variable and make all occurrences of that variable as 1, write minterms (1's) as don't cares, leave 0's and don't cares as it is. Now, obtain the SOP expression.
- Step 3: Multiply the obtained SOP expression with the concerned variable.
- Step 4: Repeat step 2 for all the variables in the k-map.
- Step 5: SOP of VEM is obtained by ORing all the obtained SOP expressions.

Previous



Aspects	Combinational Circuit	Sequential Circuit
Definition	Output depends only on present inputs.	Output depends on present inputs and past states.
Mathematical Model	Boolean expressions (pure logic functions).	Finite State Machine (FSM) with transition functions.
Memory Element	No memory; does not retain past states.	Requires memory elements like flip-flops.
Clock Dependency	Operates asynchronously.	Operates synchronously with a clock signal.
Feedback Loop	No feedback path; purely feedforward logic.	Has feedback paths that influence future outputs.
Power Consumption	Lower, as it operates only when inputs change.	Higher due to continuous clocking and state retention.

Delay Considerations	Propagation delay depends on gate delays only.	Propagation delay depends on clock cycle time and flip-flop setup/hold times.
Design Complexity	Less complex; only involves logic gates.	More complex due to state transitions and memory elements.
Reliability & Noise Sensitivity	More reliable since it does not rely on previous states.	More sensitive to noise due to clock and storage elements.
Design Techniques	Boolean algebra, Karnaugh maps (K-maps).	State diagrams, state tables, flip-flop excitation tables.
Examples	ALU (Arithmetic Logic Unit), Encoders, Decoders, Adders, Multiplexers.	Counters, Shift Registers, Latches, Flip-Flops, RAM.

Examples

CDE

AB

	000	001	011	010	110	111	101	100
00	1 ⁰	0 ¹	1 ³	0 ²	0 ⁶	1 ⁷	0 ⁵	1 ⁴
01	1 ⁸	0 ⁹	0 ¹¹	0 ¹⁰	1 ¹⁴	0 ¹⁵	0 ¹³	1 ¹²
11	1 ²⁴	0 ²⁵	0 ²⁷	1 ²⁶	0 ³⁰	0 ³¹	0 ²⁹	1 ²⁸
10	1 ¹⁶	0 ¹⁷	1 ¹⁹	0 ¹⁸	0 ²²	1 ²³	0 ²¹	1 ²⁰

Examples

$$\begin{aligned} F(A, B, C, D, E) = & A'B'C'D'E' + A'B'C'D'E + A'B'C'DE' + A'B'C'DE \\ & + A'BC'D'E + A'BC'DE + A'BCD'E + A'BCDE \\ & + AB'C'D'E' + AB'C'D'E + AB'C'DE + AB'CDE \\ & + ABC'D'E + ABC'DE + ABCDE \end{aligned}$$

$$\begin{aligned} F(A, B, C, D, E) = & A'B'C'D'E' + A'B'C'D'E + A'B'C'DE' + A'B'C'DE \\ & + A'B'C'DE + A'B'CD'E + A'B'CD'E + A'B'CD'E \\ & + A'BC'D'E + AB'C'D'E + AB'CDE + AB'CDE \\ & + ABC'DE + ABCDE \end{aligned}$$

Examples

$$F(A, B, C, D, E) = \Sigma m(4, 5, 6, 7, 20, 21, 22, 23) + d(0, 1, 2, 3)$$

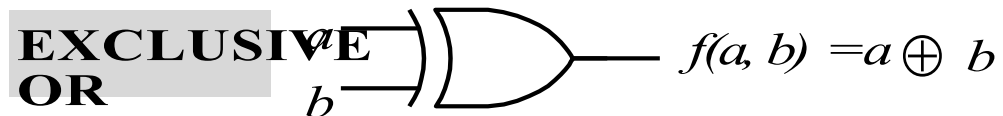
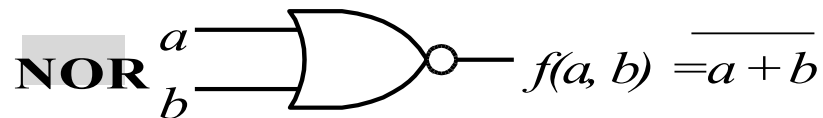
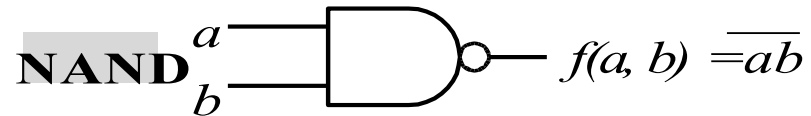
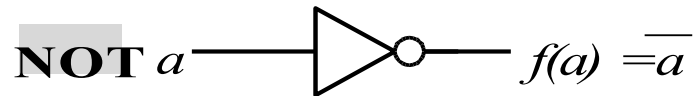
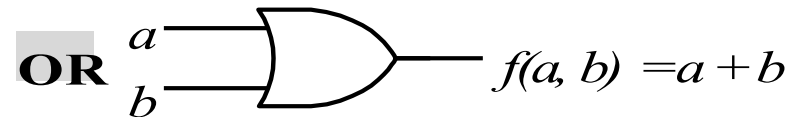
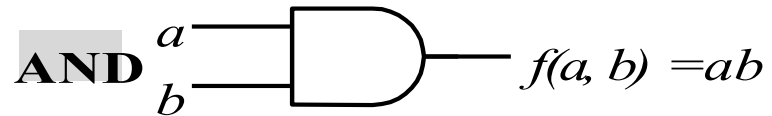
$$F(A, B, C, D, E) = \Sigma m(0, 1, 2, 3, 8, 9, 10, 11, 16, 17, 18, 19)$$

$$F(A, B, C, D, E) = \Sigma m(15, 23, 27, 29, 30, 31)$$

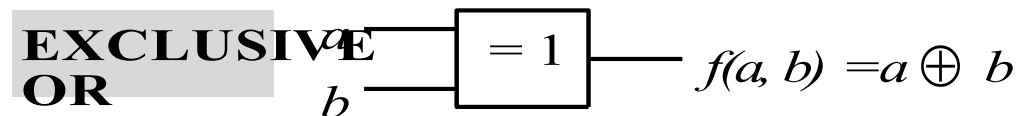
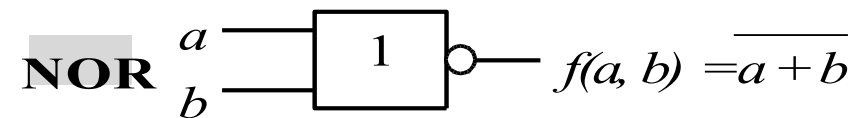
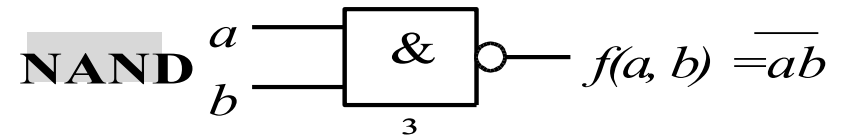
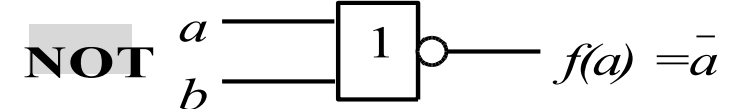
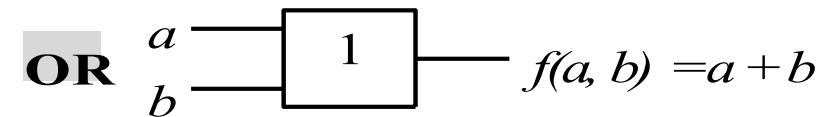
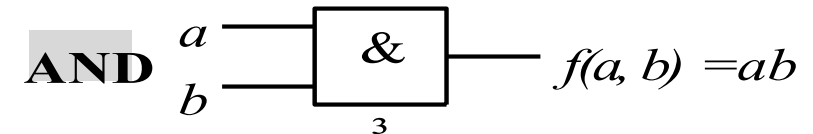
$$F(A, B, C, D, E) = \Sigma m(1, 5, 6, 7, 14, 15, 16, 20, 22, 23, 30, 31)$$

$$F(A, B, C, D, E) = \Sigma m(0, 1, 4, 8, 12, 13, 15, 16, 17, 23, 29, 31)$$

Electronic Logic Gates (2)



Symbol set 1



Symbol set 2
(ANSI/IEEE Standard 91-1984)

Active High and Active Low

In digital circuits and logic design, the terms active low and active high refer to the logic levels that signify an "active" state for a signal or a component, typically in the context of inputs, outputs, or control signals.

Active High

Definition: A signal is considered active high when a high voltage level (usually represented as '1' or logic high) indicates the active state of the signal.

Example: In a circuit, if a button is pressed and it outputs a high voltage (e.g., +5V), that state is interpreted as "on" or "active." For instance, a logic gate that outputs high when its inputs are satisfied is said to be active high.

Active High and Active Low

Active Low

Definition: Conversely, a signal is considered active low when a low voltage level (usually represented as '0' or logic low) indicates the active state of the signal.

Example: In this case, pressing a button might pull the signal to ground (0V), which is interpreted as "on" or "active." Many reset signals in digital circuits are active low, meaning they are triggered when the signal is low.

Active High: Active state = high voltage (1)

Active Low: Active state = low voltage (0)

Active High and Active Low

Positive Versus Negative Logic

	Positive Logic	Negative Logic
1 is represented by	High Voltage	Low Voltage
0 is represented by	Low Voltage	High Voltage

Active High and Active Low

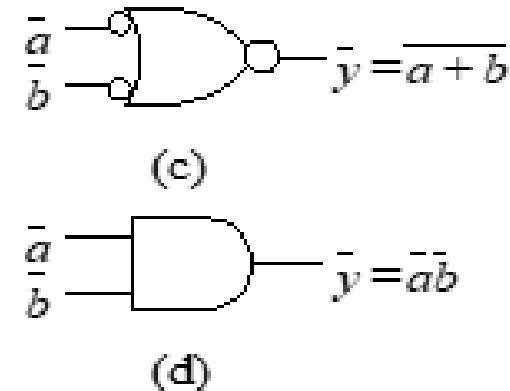
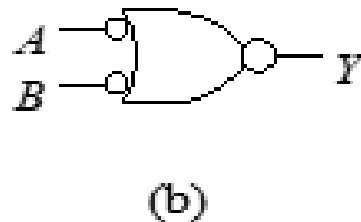
Inversion: When using active low signals, a NOT gate may be used to invert the signal for logic operations, while active high signals operate without inversion.

The distinction between active high and active low is fundamental in digital logic design, influencing how devices are controlled and how signals are interpreted within a circuit.

- *AND Gate Usage in Negative Logic*

A	B	Y
1	1	1
1	0	1
0	1	1
0	0	0

(a)

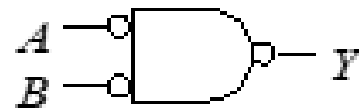


- (a) AND gate truth table ($L = 1, H = 0$)
- (b) Alternate AND gate symbol (in negative logic)
- (c) Preferred usage
- (d) Improper usage
- $y = a \cdot b = \overline{\overline{a \cdot b}} = \overline{\overline{a} + \overline{b}} = \bar{f}_{OR}(\bar{a}, \bar{b})$
- $\bar{y} = \overline{(\bar{a}) + (\bar{b})} = \overline{\bar{a} + \bar{b}} = \bar{f}_{OR}(a, b)$

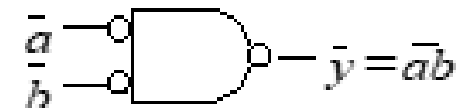
- *OR Gate Usage in Negative Logic*

A	B	Y
1	1	1
1	0	0
0	1	0
0	0	0

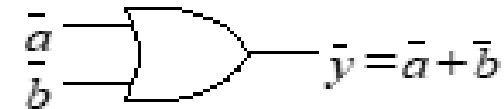
(a)



(b)



(c)

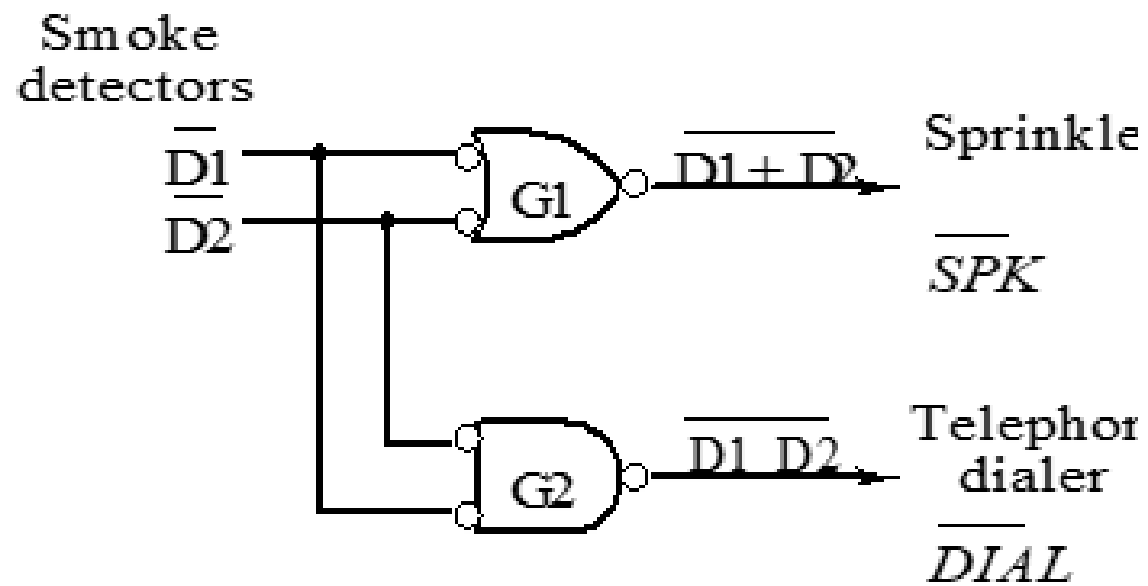


(d)

- (a) OR gate truth table ($L = 1, H = 0$)
- (b) Alternate OR gate symbol (in negative logic)
- (c) Preferred usage
- (d) Improper usage
- $y = a + b = \overline{\overline{a + b}} = \overline{\overline{a} \cdot \overline{b}} = \bar{f}_{AND}(\bar{a}, \bar{b})$
- $\bar{y} = \overline{(\overline{a}) \cdot (\overline{b})} = \overline{\overline{a} \cdot \overline{b}} = \bar{f}_{AND}(a, b)$

- *Example 2.32*: Building smoke alarm system
 - Components: two smoke detectors, a sprinkler, and an automatic telephone dialer
 - Behavior:
 - Sprinkler is activated if either smoke detector detects smoke.
 - When both smoke detector detect smoke, fire department is called.
 - Signals:
 - $\overline{D1}, \overline{D2}$: Active-low outputs from two smoke detectors.
 - $\overline{SPK}$: Active-low input to the sprinkler
 - $\overline{DIAL}$: Active-low input to the telephone dialer.
 - Logic equations
 - $\overline{SPK} = \overline{D1 + D2}$
 - $\overline{DIAL} = \overline{D1 \cdot D2}$

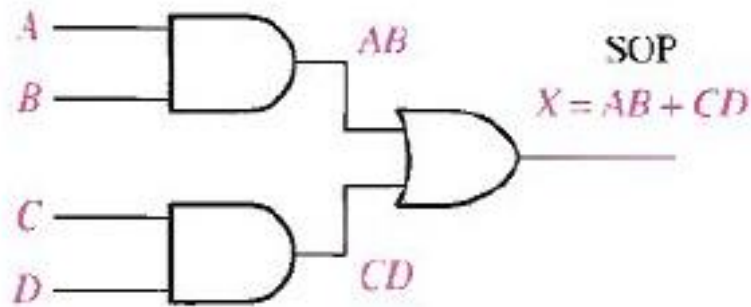
Logic diagram of the smoke alarm system



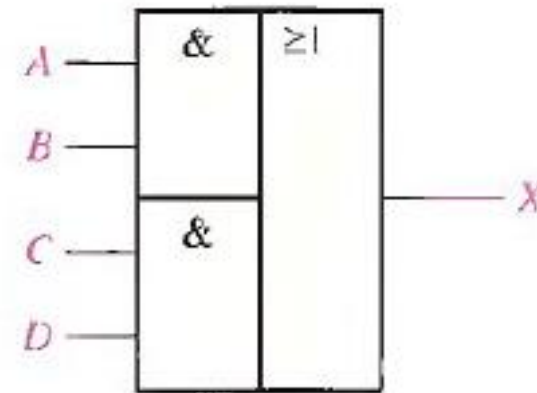
Combinational Logic Circuit Analysis

1. AND-OR Logic

The Boolean expressions for the AND gate outputs and the resulting SOP expression for the output X are shown in the diagram. In general, all AND-OR circuit can have any number of AND gates each with any number of inputs.



(a) Logic diagram



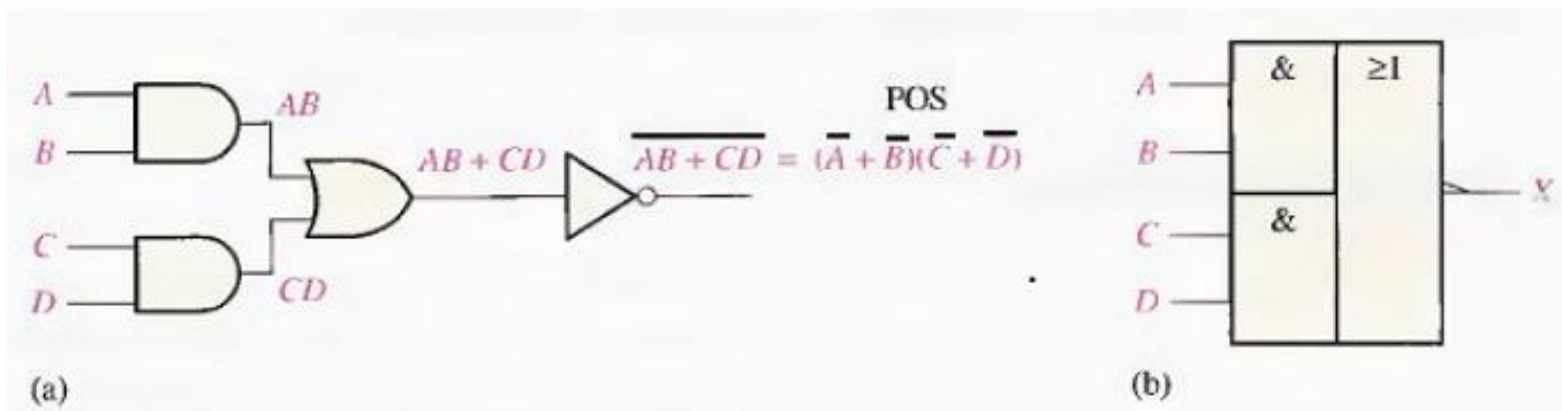
(b) ANSI standard rectangular outline symbol.

Combinational Logic Analysis

2. AND-OR Invert Logic

When the output of an AND-OR circuit is complemented (inverted), it results in an AND-OR-Invert circuit. POS expressions can be implemented with AND-OR-Invert logic. This is illustrated as follows, starting with a POS expression and developing the corresponding AND-OR-Invert expression.

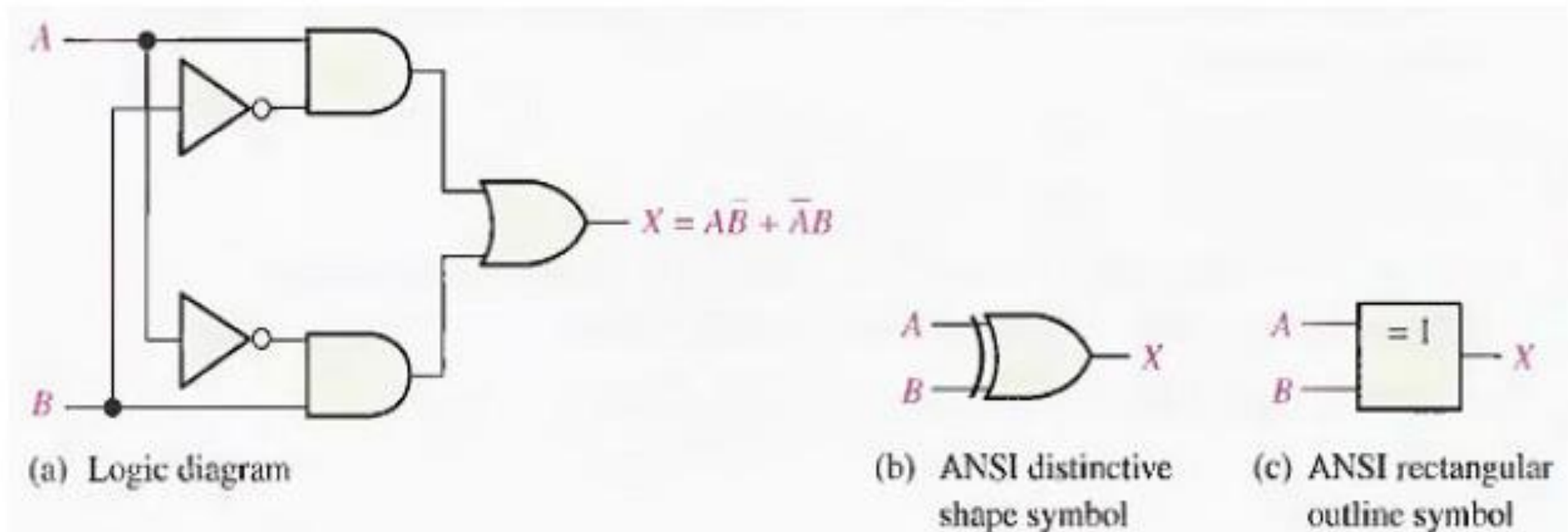
$$X = (\bar{A} + \bar{B})(\bar{C} + \bar{D}) = (\overline{AB})(\overline{CD}) = \overline{\overline{\overline{AB}}(\overline{CD})} = \overline{\overline{AB} + \overline{CD}} = \overline{AB + CD}$$



Combinational Logic Analysis

3-Exclusive-OR logic

The exclusive-OR gate was introduced before. Although, because of its importance, this circuit is considered a type of logic gate with its own unique symbol it is actually a combination of two AND gates, one OR gate, and two inverters, as shown here:



Combinational Logic Analysis

4- Exclusive-NOR Logic

As you know, the complement of the exclusive-OR function is the exclusive-NOR, which is derived as follows:

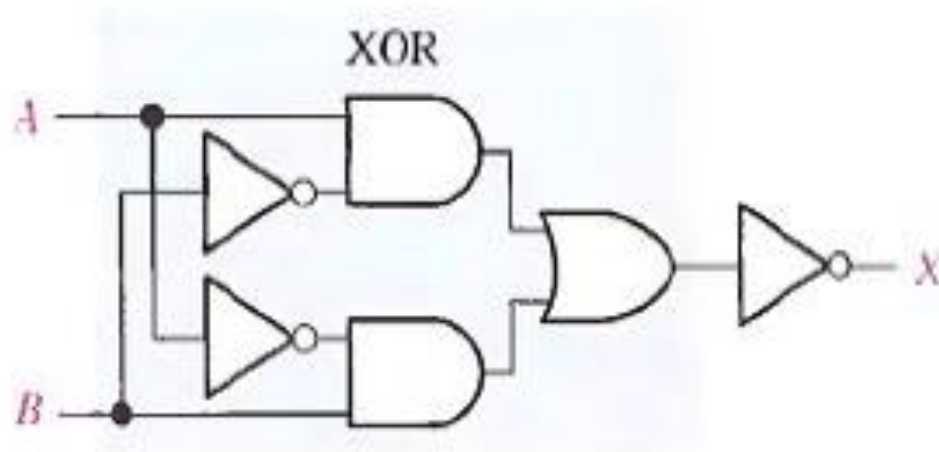
$$X = \overline{AB + \overline{A}\overline{B}} = \overline{(AB)} \overline{(\overline{A}\overline{B})} = (\overline{A} + \overline{B})(A + B) = \overline{A}B + A\overline{B}$$

Notice that the output X is HIGH only when the two inputs, A and B, are at the same level. The exclusive-NOR can be implemented by simply inverting the output of an exclusive-OR, as shown in figure in the next slide, or by directly implementing the expression $\overline{A}B + A\overline{B}$.

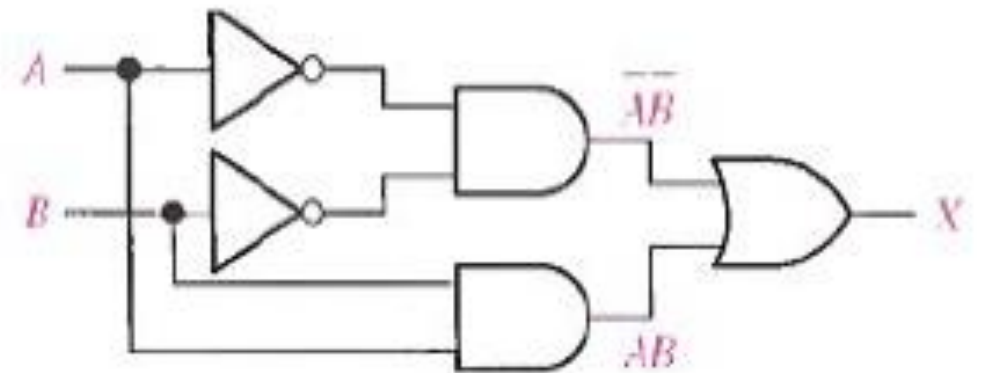
Combinational Logic Analysis

4- Exclusive-NOR Logic

The complement of exclusive OR is Exclusive NOR. Notice that output is high only when two inputs are at the same level.



(a) $X = \overline{A\overline{B}} + \overline{\overline{A}B}$



(b) $X = \overline{A\overline{B}} + \overline{\overline{A}B}$

Combinational Logic Analysis

4- Exclusive-NOR Logic

Example

Develop a logic circuit with four input variables that will only produce a 1 output when exactly three input variables are 1s.

Universal Property of NAND

1- The NAND Gate as a Universal Logic Element

NAND Definition: A NAND gate is an AND gate followed by a NOT gate.

Truth Table:

A	B	A NAND B
0	0	1
0	1	1
1	0	1
1	1	0

Basic Implementations:

- NOT using NAND: $A \text{ NAND } A = \text{NOT } A$
- AND using NAND: $(A \text{ NAND } B) \text{ NAND } (A \text{ NAND } B)$
- OR using NAND: $(A \text{ NAND } A) \text{ NAND } (B \text{ NAND } B)$

Universal Property of NAND

1- The NAND Gate as a Universal Logic Element

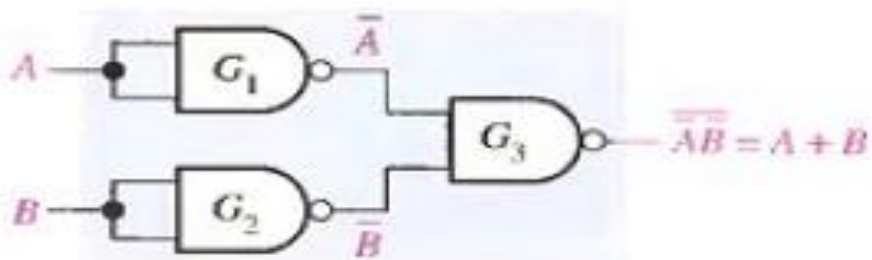
The NAND gate is a universal gate because it can be used to produce the NOT, the AND, the OR, and the NOR functions. An inverter can be made from a NAND gate by connecting all of the inputs together and creating, in effect, a single input, as shown in Fig. (a) for a 2-input gate. An AND function can be generated by the use of NAND gates alone, as shown in Fig.(b). An OR function can be produced with only NAND gates, as illustrated in part (c). Finally, a NOR function is produced as shown in part (d).



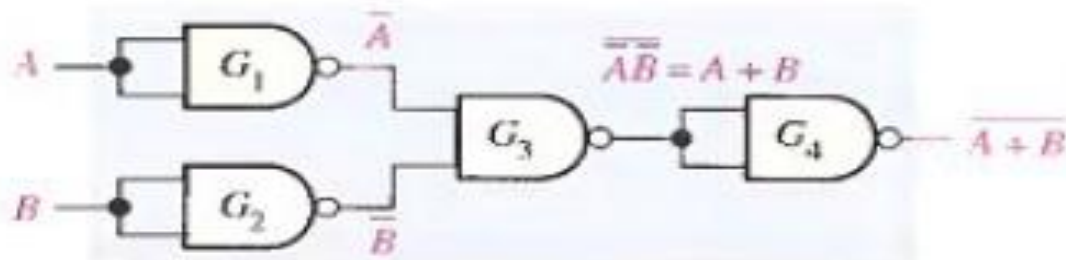
(a) One NAND gate used as an inverter



(b) Two NAND gates used as an AND gate



(c) Three NAND gates used as an OR gate



(d) Four NAND gates used as a NOR gate

Universal Property of NOR

2- The NOR Gate as a Universal Logic Element

NOR Definition: A NOR gate is an OR gate followed by a NOT gate.

Basic Implementations:

- NOT using NOR: $A \text{ NOR } A = \text{NOT } A$
- OR using NOR: $(A \text{ NOR } B) \text{ NOR } (A \text{ NOR } B)$
- AND using NOR: $(A \text{ NOR } A) \text{ NOR } (B \text{ NOR } B)$

A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0

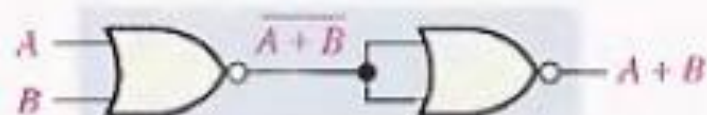
Universal Property of NOR

2- The NOR Gate as a Universal Logic Element

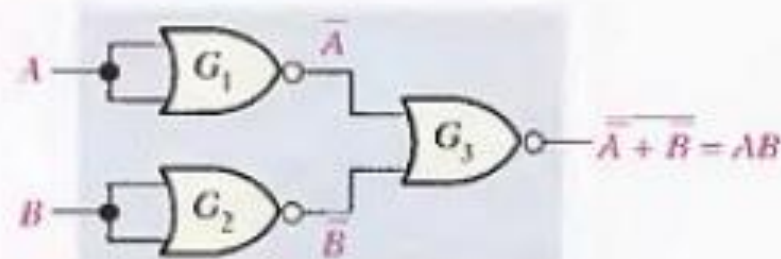
Like the NAND gate, the NOR gate can be used to produce the NOT, AND, OR and NAND functions. A NOT circuit, or inverter, can be made from a NOR gate by connecting all of the inputs together to effectively create a single input, as shown in Fig.(a) with a 2-input example. Also, an OR gate can be produced from NOR gates, as illustrated in Fig.(b). An AND gate can be constructed by the use of NOR gates, as shown in Fig.(c). In this case the NOR gates G 1 and G 2 are used as inverters. Fig (d) shows how NOR gates are used to form NAND function.



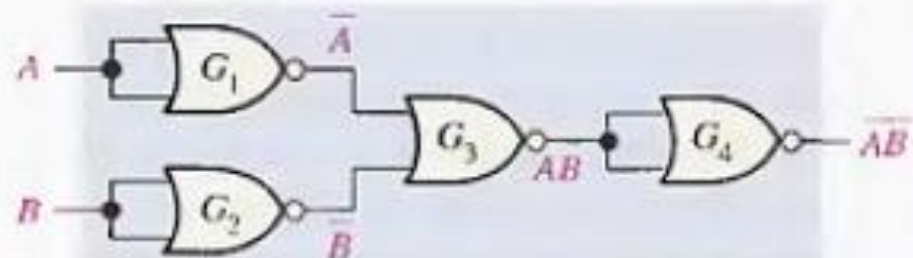
(a) One NOR gate used as an inverter



(b) Two NOR gates used as an OR gate



(c) Three NOR gates used as an AND gate



(d) Four NOR gates used as a NAND gate

NAND and NOR implementation

Feature	NAND Implementation	NOR Implementation
Number of Gates	Generally fewer	Slightly more
Speed	Faster in some cases	May introduce delay
Common Usage	Widely used in microprocessors	Used in memory design

NAND and NOR implementation

Step 1: Write the Boolean Expression

Step 2: Convert Expression to SOP Form

Step 3: Whenever the plus or dot operator; apply Double Negation; expand the inner bar.

Step 4: Apply De Morgan's Theorem

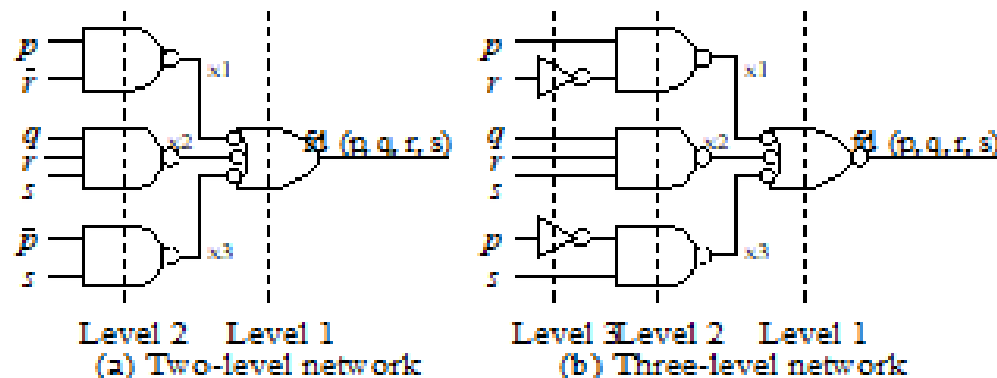
Step 5: Replace Gates with NAND Gates

Result: Circuit uses only NAND/NOR gates

Synthesis of Combinational Logic Circuit

Two-level Circuits

- Input signals pass through two levels of gates before reaching the output.



- Implementation procedure for NAND (NOR) logic:
 - Step 1. Express the function in minterm (maxterm) list form.
 - Step 2. Write out the minterms (maxterms) in algebraic form.
 - Step 3. Simplify the function in SOP (POS) form.
 - Step 4. Transform the expression into the NAND (NOR) form.
 - Step 5. Draw the NAND (NOR) logic diagram.

Combinational Circuits using TTL 74XX ICs

IC No.	Description
7400	Quad 2-input NAND gates
7402	Quad 2-input NOR gates
7404	Hex Inverter (6 NOT gates)
7408	Quad 2-input AND gates
7410	Triple 3-input NAND gates
7411	Triple 3-input AND gates
7420	Dual 4-input NAND gates
7421	Dual 4-input AND gates
7427	Triple 3-input NOR gates
7430	8-input NAND gate
7432	Quad 2-input OR gates
7486 / 74386	Quad 2-input EX-OR (XOR) gates
74133	13-input NAND gate
74135	Quad EX-OR / EX-NOR gates
74260	Dual 5-input NOR gates

Summary

Active High:

- A signal or pin is considered "active high" when its high state (typically represented by a logic '1' or a higher voltage level) corresponds to the active or "ON" condition of a device or function.
- When the signal is high, the associated action or operation is enabled or triggered.
- When the signal is low (logic '0' or lower voltage), the action is inactive or "OFF".

Active Low:

- A signal or pin is "active low" when its low state (logic '0' or lower voltage) signifies the active or "ON" condition.
- In this case, a low signal triggers the associated action or operation.
- A high signal (logic '1' or higher voltage) indicates an inactive or "OFF" state.

Summary

NAND Implementation:

NAND gates are universal gates, meaning they can be used to implement any other logic function. To implement other logic functions using NAND gates, you need to use a combination of NAND gates to achieve the desired output. This involves using NAND gates to create inverters (NOT gates), AND gates, and OR gates.

For example, to create an AND gate using NAND gates, you can invert the outputs of two NAND gates. To create an OR gate, you can invert the inputs of two NAND gates and then combine their outputs in another NAND gate.

Summary

NOR Implementation:

Similar to NAND gates, NOR gates are also universal gates.

- You can implement other logic functions using a combination of NOR gates.
- To create an inverter (NOT gate) with a NOR gate, you can simply connect the input to both inputs of the NOR gate.
- To create an AND gate, you can invert the outputs of two NOR gates.
- To create an OR gate, you can invert the inputs of two NOR gates.

Thank you!